QUALITY ASSURANCE TESTING DOCUMENTATION (QATD)

What group name?

Table of Content

1. Scope & Requirements Traceability 3
    1.1 In-Scope Core Features 3
    1.2 Out-of-Scope 3
2. Risk Assessment & Mitigation Strategy 4
3. Test Environment & Execution Strategy 6
4. CI/CD Release Thresholds & Automation Gates 7
5. Test Case Specifications (Drafts) 8
6. AI Output & Boundary Testing (Drafts) 10

Document Control

Field	Detail
System Under Test (SUT)	Zenius
Team Repo URL	https://github.com/JY156/UMHackathon_Zenius.git
Project Board URL	https://github.com/users/JY156/projects/1

Objective:

To ensure Zenius accurately ingests unstructured data to detect team disruptions (leave/burnout) and performs multi-step reasoning to reassign tasks without exceeding load thresholds.

1. Scope & Requirements Traceability

The purpose of this section is to align testing efforts with the core functional requirements, ensuring every planned feature is validated and that "feature creep" is minimized.

1.1 In-Scope Core Features

- **Task Extraction & Ingestion:** Ingesting unstructured data from Slack, emails, and meeting transcripts using the FastAPI sensor layer.
- **AI Classification:** Utilizing the Z.AI GLM to perform the five-step orchestration (Detect → Analyze → Match → Check→ Execute).
- **Proprietary Burnout Detection:** Calculating "Load Scores" based on task volume, communications tone (sentiment analysis), and irregular working hours (12 AM – 5 AM).
- **Dynamic Workflow Orchestration:** Automated reassignment of tasks in Firebase and generating "Resilience Reports" for leaders and "Workload Adjustments" for members.
- **Agentic Feedback Loops:** The ability of the GLM to re-reason and find a second-best option if a reassigned member rejects a task.

1.2 Out-of-Scope

- **Direct External Integration:** Integration with payroll systems or external client-facing portals.
- **User Profile Customization:** Detailed UI features like theme selection, profile picture uploads, or notification sounds.
- **Historical Data Migration:** Importing large-scale historical project data from external legacy systems.
- **Real-time Voice Synthesis:** Using AI to speak back to users during meetings (text-based transcripts only).

2. Risk Assessment & Mitigation Strategy

Technical Risk	Likelihood	Severity	Risk Score	Mitigation Strategy	Testing Approach
GLM Hallucination: AI reassigns a task to a member already at high load.	3	5	**15 (High)**	Implement strict "Load Score" checks (>80) in the reasoning.py logic.	Edge Case testing with "High Load" mock profiles.
API Rate Limiting: Z.AI GLM API becomes unresponsive during peak "noise".	4	4	**16 (Crit)**	Implement a "Graceful Degradation" state where the system pings the leader manually.	Simulate API timeout (5s) and verify fallback UI.
Data Conflict: Email and PM tool provide contradictory status.	2	3	**6 (Med)**	GLM acts as "Conflict Resolver" to ping user for status.	Negative test: Input conflicting JSON fields.

Risk Assessment Scoring Criteria:

Likelihood (1-5)		*Severity (1-5)*	
1	*Rare*	*1*	*Impact is Negligible*
2	*Unlikely*	*2*	*Impact is Minor*
3	*Possible*	*3*	*Moderate Impact*
4	*Likely*	*4*	*Major Impact*

5	*Almost Certain*	*5*	*Critical Failure of the system*

Risk Score = Likelihood × Severity

Risk Level Reference:

Risk Score	***Risk Level***	***Recommended Action***
1 – 5	*Low*	*Monitor only. Acceptable risks.*
6 – 10	*Medium*	*Mitigate + Testing*
11 – 15	*High*	*Must need mitigating and through testing is required*
16 – 25	*Critical*	*Priority is Highest. Need extensive level of testing.*

3. Test Environment & Execution Strategy

Explain the testing environment. That means where the testing of your application will take place, how you are going to handle test data and the rules/threshold that you are incorporating to govern your testing phases.

- **Unit Test**
    - Scope: Validation of the Burnout Detection logic and the "Conflict Resolver" state in reasoning.py.
    - Execution: Tests are written using the PyTest framework; execution occurs locally during development and automatically via the GitHub Actions CI pipeline upon repository push.
    - Isolation: To isolate the reasoning logic, external dependencies such as the Firebase Firestore database and the Z.AI GLM API are mocked to ensure tests remain fast and deterministic.
    - Pass Condition: Success is defined by passing all happy cases (valid reassignment), negative cases (blocking assignment to an overloaded member), and edge cases (detecting the 12 AM – 5 AM high-risk flag).

- **Integration Test**
    - Scope: Communication between the FastAPI Sensor Layer and the Node.js/Express Action Layer.
    - Execution: Performed after unit tests pass, specifically when merging feature branches (e.g., sensor-logic) into the develop branch.
    - Workflow: Real API calls are tested without mocking to ensure the Firebase Approvals collection is correctly updated with the {suggestedTid, fromUid, toUid, reasoning} JSON output.
    - Pass Condition: Integration is successful when a detected "Sick Leave" trigger in the sensor successfully creates a pending approval record in Firebase.

- **Test Environment (CI/CD Practice):**
    - Local Testing: Initial manual and automated verification conducted on localhost:3000 (Frontend) and localhost:8000 (FastAPI).
    - Staging/CI: GitHub Actions serves as the automation gate, triggering the test suite on every push to prevent code regressions.
    - Automated Pipeline: Basic linting and unit tests are run on branch levels, while full integration tests are required for any Pull Request (PR) directed at the main branch.

- **Regression Testing & Pass/Fail Rules:**
    - Execution Phase: Full suite testing is conducted across the three core features (Absence Handling, Burnout Prevention, and Task Extraction) whenever a new branch is merged to main.
    - Pass/Fail Condition: A test is "Passed" only if the actual system output perfectly matches the expected orchestration result; any mismatch is logged in the Defect Log and blocks the release.
    - Continuation Rule: Testing follows a "Gradual Gate" logic where initial unit tests must meet a 100% threshold before integration or regression testing can commence.

- **Test Data Strategy:**
    - Manual Data: Created 5 distinct Mock User profiles in Firebase with varying "Load Scores" and "Sentiment Scores" to simulate different burnout states.
    - Automated Data: Utilizing the node backend/seedDatabase.js script to rapidly populate the environment with dummy tasks and communication logs for stress testing.

- **Passing Rate Threshold:**
    - General Threshold: A minimum 80% pass rate is required for all non-critical test cases.
    - Critical Threshold: 100% pass rate is mandatory for "Critical" features, specifically Burnout Prevention (ensuring no over-assignment) and Absence Handling (ensuring no orphaned tasks).
    - Justification: As Zenius manages human resiliency, any failure in burnout detection can lead to severe organizational fatigue in a real-world production environment.

4. CI/CD Release Thresholds & Automation Gates

4.1 Integration Thresholds (Merging to Main)

The integration threshold is crucial to ensure that the main branch remains a stable, production-ready environment. This is achieved through strict Continuous Integration (CI) checks before any pull request is merged.

Checks	**Requirements**	**Project**	**Pass/Failed**
Automatic Build	**Zero Build Error**	**0 Errors**	**Passed**
Unit Tests	**100% Passing Rate**	**100%**	**Passed**
Code Quality	**Zero Linting Error**	**100% (Clean)**	**Passed**
Test Coverage	**Minimum 81.2%**	**85%**	**Passed**

4.2 Deployment Thresholds (Pushing to Production)

Checks	Requirements	Project	Pass/Failed
Regression Test	Minimum 90%	93%	Passed
AI Output Pass Rate	Minimum of 80%	85%	Passed
Critical Bugs	Zero P0/01 Bugs	0	Passed
API Performance	Response Time < 800ms	420ms	Passed
Security	API keys are not exposed	Clean (via .env)	Passed

5. Test Case Specifications (Drafts)

Test Case ID	Test Type & Mapped Feature	Test Description	Test Steps	Expected Result	Actual Result

TC-01	**Happy Case (Entire Flow):** Absence Orchestration	Verify the "Golden Path": System detects a "Sick" trigger and reassigns an orphaned task to a peer.	1. Access Dashboard as Project Leader. 2. Input Slack log: "I'm sick, won't be in." 3. GLM identifies orphaned tasks. 4. GLM matches skills and load score. 5. Execute reassignment.	Task re-assigned in Firebase; "Resilience Report" generated for Leader; "Workload Adjustment" sent to peers.	Successfully detected leave trigger. GLM matched "Frontend" skill to User_B (Load: 45). Firebase updated in 1.5s.
TC-02	**Specific Case (Negative):** Burnout Prevention Block	Verify the system blocks reassignment to a member with a **Load Score > 80** or a **"Stressed" tone**.	1. Set User_C Load Score to 85. 2. Trigger task reassignment. 3. GLM attempts to assign to User_C.	GLM detects high load (>80); blocks assignment; re-reasons to find a second-best option.	Engine successfully flagged User_C as "Burnout Risk." Task diverted to User_D. No 500 errors triggered.

TC-03	**NFR (Performance):** Multi-step Reasoning Latency	Verify the GLM multi-step reasoning process remains responsive ($< 3000ms$) under load.	1. Initiate full orchestration flow via API call. 2. Measure time from "Input Ingestion" to "Action Output."	Average response time for the reasoning logic should be < 3000ms (3 seconds).	Average Multi-step reasoning time recorded as 2450ms. 0% Error Rate.
TC-04	**NFR (Reliability):** State Persistence	Verify that a failed GLM API call does not result in orphaned/corrupted data tasks.	1. Force a Z.AI API timeout (simulate failure). 2. Check Firebase Task status.	System pings Leader for "Conflict Resolution" or manual update; no data is corrupted.	Fail-safe triggered. System displayed "Awaiting AI Reasoning" status rather than crashing.

6. AI Output & Boundary Testing (Drafts)

This section is designed to validate that your GLM integration does produce acceptable output and handles any inputs that are abnormal gracefully.

6.1. **Prompt/Response Test Pairs**

Test ID	Prompt Input	Expected Output (Acceptance Criteria)	Actual Output	Status
AI-01: Intent Extraction	"I'm feeling completely	Must identify "Burnout	Reassign the task to other member,	Pass

	overwhelmed and need to take a break."	Warning" and suggest a task cool-down period.	and ask for approval from manager	
AI-02: Structured Action	"Extract tasks from meeting minutes: 'John to finish the UI by Friday.'	Output JSON must contain {task: "Finish UI", assignee: "John", deadline: "Friday"}.	Task: Finish UI Assignee: John Deadline: Friday	Pass

6.2. Oversized/Larger Input Test

Fields	Details
Maximum Input Size	2000 Tokens
Expected Behavior	System must chunk transcripts into segments before reasoning to avoid token overflow
Actual Behavior	Able to process input
Status	Pass

6.3. Adversarial/Edge Prompt Test

Test ID	**Ambiguous/Malformed Input**	**Expected System Behavior**	**Actual GLM Response**	**Status**

AI-03	"I'm not sure if I can make it, everything is just a mess right now."	The system should identify high emotional distress but ambiguous intent. It must trigger a "Conflict Resolver" status ping to the user for clarification rather than reassigning tasks immediately.	GLM flagged "High Stress" but "Undetermined Intent." System sent a clarification ping to the user.	**Passed**

6.4. Hallucinating Handling

To mitigate the risk of the GLM providing inaccurate or "hallucinated" reassignments (e.g., assigning a task to a non-existent member or ignoring load constraints), Zenius implements the following technical safeguards:

- **Strict JSON Enforcement:** The system uses a "Constraint-Based" prompting strategy that forces the GLM to output ONLY valid, schema-compliant JSON. If the output is conversational or malformed, the **Action Layer** (Node.js) rejects the payload before it reaches the database.
- **Load-Score Validation (Guardrails):** Even if the GLM suggests a reassignment, the **Reasoning Engine** performs a secondary "Hard Check" against the Firebase User collection. If the suggested user's load is actually >80, the system overrides the AI's decision and triggers a "Re-reasoning" loop.
- **Audit Trail Reasoning:** Every output from the GLM must include a reasoning field. This provides transparency for the Project Leader to verify the logic behind a reassignment, allowing for a manual override if a hallucination is detected.

Important Note: Agile Testing Principles

1. ***Sprint-Based Validation:*** *Tests are run during each feature build (e.g., testing the "Conflict Resolver" immediately after the sensor layer is coded).*
2. ***Continuous Feedback:*** *Failures in* ***AI Output Pass Rates*** *result in immediate prompt refactoring or logic adjustments in reasoning.py.*
3. ***Automated Gates:*** *Every code push triggers the* ***GitHub Actions*** *suite to ensure that new logic does not break the previously passed "Happy Case" flows.*